Building Web and Mobile Apps WS2025/26 (Prof. Dr. Cathrin Möller)

Exercise 4 - 17./18.11.2025

Do not hesitate to ask questions or if you want feedback for your coding style.

Please commit and push to GitLab regularly and ensure @*cathrin.moeller* and my two tutors @pinky-jitendra.tinani and @rahul.patil are invited as a reporter.

Learning goals for today

Before we deep dive into Angular, the goal is today to get at least a hello world experience using other frameworks like React or Vue. In case you are very familiar with React already, feel free to create a new App in Vue with the same features as described for React.
We continue connecting frontend and backend as this is a crucial development task.
In our simple React Hello World example we will use the fetch API (axios would be an alternative framework). We want to load data, parse it, add a button event, delay the backend response and show an animated spinner while waiting.

Practical Exercises

1) Create a sub-folder for the react app in your GitLab repo that you can submit. Maybe you have a vanilla or frontend folder already next to the backend folder for the expressJS app. Create a react folder next to them. We can get a **simple React startup boilerplate** using the command line tool „create-react-app“. We can install it via NPM or just execute (without installing) using NPX. Run `npx create-react-app my-react-app` and start your app via `cd my-react-app` and then `npm start`. Open the hello world startup app. What do you see? What is the URL (including port number) the frontend is running?

1b) If you worked a lot with React already, create a folder in your GitLab for „vue“ and execute „npm create vue@latest“ to create a Vue Hello World app. Just press enter for all questions to start with default settings. Visit https://vuejs.org/guide/quick-start.html for more info. After running „npm install“ you

can run „npm run dev“ to see the app running in your browser. What do you see? What is the URL (including port number) the frontend is running?

2) Frameworks like React do a lot of complex „magic“ internally, simplifying development compared to „vanilla coding“. Use the browser dev tools and the network tab to understand how many HTML, CSS and JS files are loaded in order to render the app. Add a markdown file BWMA.md and note down your findings.

3) Let’s change the content in the App.js (React) component to perform a fetch call.
Add this code here:

```
import React, { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [message, setMessage] = useState('');

  useEffect(() => {
    fetch('http://localhost:8081/api/message')
      .then(response => response.json())
      .then(data => setMessage(data.message))
      .catch(err => console.error("Failed to fetch message:", err));
  }, []);

  return (
    <div className="App">
      <header className="App-header">
        <p>
          Message from backend: {message}
        </p>
      </header>
    </div>
  );
}

export default App;
```

Run your app. Ensure the express JS backend is running using port 8081 and add an endpoint to answer the /api/message GET request with a JSON response that allows to be parsed receiving the message. You can debug

using the browser and testing the URL. Make sure your CORS settings allow the new react frontend URL to receive data!

4) Now rewrite the template in React to **show the data from the expressJS backend** from last weeks that forwarded a **list of users and addresses** from a hardcoded JSON file. You will need another fetch API call for the users data. Add another HTML node underneath the message to not lose the results from the previous exercise. (Might be needed to change the CSS or to add the node inside the <header> that the content is visible.)
Ensure the **mapping in your frontend is right to parse the data correctly**.
Hint: in React you can use a syntax like this to show a list of entries:

```
<ul>
  {users.map(user => (
    <li key={user.id}>
      {user.firstname}
    </li>
  ))}
</ul>
```

The unique key is needed by React for change detection and re-rendering. If your user does not have an id, add one to the JSON and the typescript interfaces. If you do not see anything, inspect the HTML and check if the content is hidden by CSS. Print the complete user info with names, email, phone and address.
Hint: React expects a single HTML node inside the return () function. So if you want to render more HTML nodes, you need to nest them.
Using ? you can ensure the rendering does not fail on undefined => {user.address?.code}

5) Now **delay the backend response with a timeout of 3 seconds** that the frontend has to wait for it. You can use the *`setTimeout()`* function for this.

6) Now change the code in the frontend to not **fetch the data** opening the app but **on button click**. A **new button click should delete the previously known data** (= reset the state in the frontend).
Hint: You can use *`<button onClick={fetchUsers}>Load Users</button>`*
If you define a fetchUsers function in your App function like this:
`const fetchUsers = () => { … }`

7) Let's create a spinner using **CSS Animations** (which are similar to transitions just running endlessly) to be shown while we wait for the backend data. The idea is to show it once you start fetching data and to hide it once you received the data from the backend.

Hint: You can add a new state variable like **loading** and use it in your App function.

const [loading, setLoading] = useState(false);

Change the value with *setLoading(true);*

In the template you can render conditional content using this syntax:

{loading ? <HTML-SPINNER/> : <HTML-NO-SPINNER/> }

A CSS animation (repeated transition) can look like this:

```
.spinner {
  border: 1px solid white;
  width: 50px;
  height: 50px;
  border-radius: 50%;
  border-left-color: gray;
  animation: spin 500ms ease infinite;
}
@keyframes spin {
  0% {
    transform: rotate(0deg);
  }
  100% {
    transform: rotate(360deg);
  }
}
```

You can play around with it adding more steps inside the animation. Imagine a different transformation at 20% / 40% / 60% and 80% e.g. You could also create a spinner with multiple circles.